Progress Report 2

Private and Verifiable Ethereum State Reads

CS6666 | Blockchain and Distributed Systems

Rohan Arora | Roll No. **CS26S022**

1. Progress Since Report 1

The project studies private and verifiable access to Ethereum contract state, using WETH balances at a fixed Sepolia snapshot. Report 1 covered the architecture, Rust foundation, workload selection and RPC provider study. The work since then has focused on real proof collection, native proof verification, record encoding and local PIR experiments.

Work completed so far. A dataset of 32 real balance proofs has been collected and verified. These records have also been retrieved through existing PIR implementations and checked again by the verifier. Helios has been selected for the light client component, and an initial connection has been tested using synthetic data. Code, retained inputs and experiment results are available in the repository.

2. Real Proof Collection and Verification

The new dataset contains 32 addresses at Sepolia block **11919648**, using the same WETH9 contract and balance mapping as Report 1. Every proof request specifies the exact block hash. A second provider reproduced the dataset. A later failure to obtain the historical state was retained as a collection failure, without switching to another block.

Verifier comparison and selection

Two Rust libraries, `alloy-trie` and `eth_trie`, were compared on the real proofs and rejection cases. Both checked all 32 records and passed 313 rejection checks per candidate. **eth_trie 0.6.1** was selected because it can recover the checked account data directly from the proof. This fits the current record format and avoids relying on account fields supplied separately by the RPC provider.

Part	Work completed
Proof dataset	32 records at one fixed block hash, reproduced through a second provider
Balance verification	24 nonzero balances and 8 proven storage absences under the supplied root
Reusable Rust verifier	Account and storage proof checks, requested key and snapshot checks, size limits and explicit errors
Workspace validation	27 passing Rust tests covering valid reads and rejection behaviour

Checks enforced by the verifier

The account proof is checked against the expected state root, and the recovered WETH storage root is used to check the requested balance. The returned value must match the value established by the proof. An absent storage entry is accepted as zero only with a valid absence proof. Missing nodes, malformed data and a wrong requested key produce errors.

The verifier is a separate component from PIR and the light client. This keeps proof checking independent of the retrieval method and leaves root authentication to the consensus component. The present real-data results remain conditional on the provider-supplied root.

3. Decisions Carried Forward

- Continue exact block-hash collection and reject unavailable historical state without a fallback.
- Use eth_trie behind the project verifier interface, with request checks and bounded proof inputs.
- Verify every record recovered through PIR before accepting its balance.
- Keep live consensus authentication separate from provider agreement and local proof verification.

4. Proof Record Encoding

The original proof responses have different sizes, while the tested PIR arrangement uses fixed size records. A binary encoder and decoder were added to compare candidate record sizes. The common WETH account proof is stored once in the snapshot information, with each record carrying its balance and storage proof. All 32 records were encoded, decoded and verified again.

Representation	Result on the current dataset
Compact JSON	4,599 to 5,545 bytes per record
Binary before padding	2,254 to 2,731 bytes per record
3,072 byte records	All records fit, with 18.05% of database bytes used for padding
4,096 byte records	All records fit, with 38.54% padding

The 3,072 byte format is the smallest tested size that holds this dataset. It is not a general limit for Ethereum proofs. Oversized records are rejected rather than shortened. Repeated parts of the storage proofs could also be stored once, with a possible 35.84% saving in raw proof bytes here. That saving still needs to be compared with indexing, padding and any extra private requests.

5. Local PIR Experiments

The experiments use a fixed version of IKPIR and its SimplePIR and FrodoPIR implementations. Two retrieval methods were compared: requesting a record by its position and looking it up by a key. In the first method, the client finds the position locally from a public list and uses PIR to hide which position it requests.

Both methods and implementations were tested with 3,072 and 4,096 byte records. All eight configurations recovered and verified the full dataset, giving **256 successful local PIR reads**. These are repeated checks of the same 32 records, not a larger dataset.

Observation	Result and implication
Private retrieval	All 32 records recovered correctly in each of eight configurations
Lookup by position	Lower setup and reply sizes than key lookup in this experiment
Per-read communication	12,504 bytes by position versus 98,720 bytes by key for 3,072 byte records
Initial setup	Significant additional client information is required before reads, and must be counted separately

What the results establish

The local runs show that private retrieval, record decoding and native proof verification work together. They do not constitute a network service or a privacy audit. Lookup by position is the next baseline to investigate, while the final PIR implementation remains open.

Setup cost is important in interpreting the communication figures. For 3,072 byte records, the smaller SimplePIR setup estimate is about 15.7 MB, compared with 109,390 bytes for the complete dataset and shared information. Setup figures are minimum estimates, not measured network downloads. The experiment therefore does not show a bandwidth advantage for PIR on this small dataset.

A failed private read must return an error. It must not trigger an ordinary RPC request for the same address, since that would reveal the selection. Network delay, concurrent clients and the complete snapshot publication path have not yet been measured or implemented.

6. Light Client Progress

Helios has been selected as the Ethereum light client. Its role is to supply an independently checked block identity and state root to the proof verifier. An initial Rust connection has been tested using saved synthetic consensus data, including checks that reject altered updates and an incorrect expected root.

The tests also keep a recently observed block separate from a finalised block. The code does not accept a newer root as finalised merely because time has passed. These are local feasibility tests. They do **not** authenticate the real Sepolia balance dataset or establish live synchronisation.

The remaining work includes a trusted starting checkpoint, live Sepolia synchronisation, support for the required network version and recovery after restarting. Helios is selected, but the precise integration method is still open.

7. Evaluation Focus and Next Work

The next question is **when private, verifiable reads become practical**. The 32-record dataset is useful for correctness checks, but is too small to judge the practical value of PIR. Increasing the dataset alone is also not enough to claim an advantage. The comparison must account for read frequency, client storage and snapshot updates.

Downloading and caching a complete snapshot once also hides later selections. It should therefore be charged once per snapshot, rather than again for every read. The planned evaluation will compare that option with PIR, including initial setup, repeated reads and the cost of moving to a new snapshot.

Next task	Purpose
Larger scale experiments	Measure setup, query cost and memory at increasing dataset sizes without weakening cryptographic settings
Proof storage comparison	Compare complete proof records with shared proof parts, including all retrieval costs
Real and artificial datasets	Separate backend scaling results from claims about real Ethereum proof sizes and updates
Live composed read	Collect fresh proofs for a block checked by Helios and test retrieval, proof verification and root authentication together

Current position. The project remains in Phase 1. The verifier and Helios choices are settled. The final PIR implementation, record size, Helios integration method and repository licence remain open. Larger-scale evaluation, network serving and checks that all published files belong to the same snapshot are pending.

Literature and Implementations Reviewed

1. **eth_trie 0.6.1.** Used for the account and storage proof verifier. Source
2. **IKPIR.** Used for the position and key lookup experiments with SimplePIR and FrodoPIR. Source
3. **Helios.** Selected for the consensus component and tested with synthetic inputs. Source
4. **SimplePIR and DoublePIR.** Background for setup and communication cost comparisons, USENIX Security 2023. Source
5. **EIP-1186.** Reference for account and storage proof response fields. Source